Chapter 7 - Summary and Sample Questions

Why Vision Transformers matter

A Vision Transformer (ViT) applies the transformer idea to images by treating an image as a **sequence of visual tokens**. The central move is to divide the image into patches and process those patches with self-attention in much the same way that a language model processes word tokens.

ViTs matter for two reasons. First, they showed that strong image understanding does not require convolution to be the only organizing principle. Second, they became a natural bridge from image classification to modern **vision-language** and **generative** systems. The same encoder idea can feed a classifier, a contrastive objective, a multimodal alignment module, or a generative backbone.

From image to token sequence

Suppose an input image has spatial size $H \times W$ and is split into non-overlapping patches of size $P \times P$. Then the number of patches is

$$N = \frac{H}{P}\frac{W}{P}.$$

Each patch is flattened and mapped into an embedding vector in $\mathbb{R}^D$. This gives a patch-token matrix

$$X_{\text{patch}} \in \mathbb{R}^{N \times D}.$$

So the key abstraction is that an image becomes a sequence of N learned token vectors. This is conceptually powerful because it lets a transformer reason over the image using the same attention machinery developed for sequences.

Patch embedding as linear projection or convolution

Patch embedding can be understood in two equivalent ways. One view is that each flattened patch is passed through a learned linear map into $\mathbb{R}^D$. Another view is that a convolution with kernel size P and stride P simultaneously **extracts non-overlapping patches** and **projects them** into D channels.

This is important conceptually because the patch embedding stage is the point where raw pixels are converted into token representations suitable for attention. It is not yet "understanding" the image at a high semantic level; it is creating the token interface on which the transformer operates.

Positional information is essential

A transformer is not inherently aware of spatial arrangement. If the model only receives patch content and no positional information, then it cannot reliably distinguish between different spatial layouts containing the same set of patches. For this reason, positional embeddings are added:

$$X' = X + P,$$

where P is a learned or fixed positional table.

In vision, positional encoding is especially important because images are spatial objects. A design choice arises: whether to use flattened 1D positions, explicit 2D structure, or relative positional information. The choice affects how well the model handles translation, spatial relations, and changes in resolution.

The class token and image-level prediction

For classification, a learnable class token is typically prepended to the patch sequence:

$$X_0 = [x_{\text{cls}}; x_1; x_2; \ldots; x_N].$$

After passing through the encoder, the final class-token representation is used as a global image summary:

$$\text{logits} = W_{\text{cls}} x_{\text{cls}}^{(L)} + b.$$

The class token is useful because it gives the network a dedicated location in the sequence for accumulating image-level information through self-attention. In bidirectional attention, the class token can attend to every patch and every patch can influence the class token.

Transformer encoder blocks in ViT

A ViT uses stacked transformer encoder blocks. Each block contains:

1. Layer normalization
2. Multi-head self-attention
3. Residual connection
4. Layer normalization
5. MLP
6. Residual connection

The attention operation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V.$$

This means that each token can dynamically gather information from other tokens according to learned relevance scores. In vision, this allows patches from distant image regions to interact directly, which is one reason ViTs can model long-range relationships effectively.

Classification versus general-purpose encoding

A ViT encoder can serve different tasks. For classification, the output head maps the final class-token state to class logits. For representation learning or multimodal learning, the encoder instead produces a general image embedding that may come from the class token, mean pooling, or another pooling mechanism.

So the encoder is often the stable core, while the **task head** changes. This is why ViTs are so reusable across settings such as classification, retrieval, CLIP-style image-text alignment, and downstream fine-tuning.

Strengths and limitations of ViTs

ViTs have several conceptual strengths. They provide flexible global interaction through attention, align naturally with transformer-based multimodal systems, and can scale effectively with data and compute. They also offer a unified architecture that works well across classification, representation learning, and generative modeling.

However, they also have important limitations. Self-attention scales quadratically with sequence length, so making patches too small can become expensive because N grows rapidly. ViTs also do not build in the same strong local inductive biases as CNNs. As a result, they often depend more heavily on data, augmentation, pretraining, or architectural refinements.

CNNs versus ViTs

CNNs build in locality, translation-related biases, and hierarchical feature extraction through convolution and pooling. ViTs replace most of that hard-wired structure with learned token interactions. This leads to a trade-off.

CNNs can be more data-efficient and naturally aligned with local image structure. ViTs can be more flexible and often stronger when trained at scale or integrated into multimodal and generative pipelines. Modern practice increasingly shows that the two ideas are not strict opposites; many successful systems mix convolutional and transformer-style design choices.

ViTs beyond classification

Vision Transformers are not only classifiers. They are now central in masked-image modeling, multimodal encoders, and image generation. In diffusion-style systems, transformer backbones can replace CNN-style U-Nets or operate in latent/tokenized spaces. This shift matters because it suggests that the transformer is not just a language architecture adapted to vision; it has become a general-purpose design pattern for perception and generation.

A useful conceptual summary is this:

$$[B, C, H, W] \to [B, N, D] \to [B, N+1, D] \to [B, D] \to [B, K].$$

The image becomes patch tokens, a class token is added, the encoder contextualizes the sequence, and a task-specific head turns the resulting representation into the desired output.

Final Exam Conceptual Questions with Sample Answers

Question 1

Consider a Vision Transformer without any class token. Is image classification still possible? Explain conceptually how and discuss one trade-off.

Sample Answer

Yes, classification is still possible. The model can aggregate the final patch tokens using mean pooling, attention pooling, or another learned global summarization method, and then apply a classifier to that global representation.

The trade-off is that without a class token, the model loses a dedicated sequence position whose role is to accumulate task-specific global information. Pooling may be simpler and more symmetric, but it may also be less explicitly specialized for classification than a learned class token pathway.

Question 2

Why do residual connections and layer normalization play an important role in a deep Vision Transformer, even though neither one directly adds visual knowledge?

Sample Answer

They play an optimization and stability role. Residual connections help preserve information across layers and make it easier to train deep stacks without destroying earlier representations. Layer normalization stabilizes the scale of activations, which makes learning more predictable.

Although they do not encode image structure directly, they are crucial for turning the transformer from a conceptual architecture into a trainable deep system. Without such mechanisms, learning can become unstable or inefficient.

Question 3

A student claims that ViTs are simply better than CNNs because they are newer and more general. Critically evaluate this claim.

Sample Answer

The claim is too simplistic. ViTs are highly flexible and have become central in many modern systems, especially when large-scale pretraining or multimodal integration is involved. They can capture global interactions elegantly and transfer well across tasks.

However, CNNs remain strong because their inductive biases are well matched to many visual problems. They can be more efficient, more data-frugal, and highly competitive when modernized. The better conclusion is that ViTs and CNNs offer different advantages, and performance depends on data scale, compute budget, and task structure.

Question 4

Why is the rise of transformer backbones in generative image models conceptually significant, rather than just an engineering curiosity?

Sample Answer

It is significant because it suggests that the transformer has become a general representation mechanism rather than a tool tied only to language. If transformer-style backbones can support both understanding and generation, then the same broad architectural principles may unify multiple kinds of perception and synthesis.

This also changes how one thinks about model design. Instead of treating classification backbones and generative backbones as fundamentally separate families, one can view them as different uses of related token-based architectures.

Question 5

Summarize the full conceptual flow of a Vision Transformer from raw image to output prediction, and identify the main design choices at each stage.

Sample Answer

The image first enters as a tensor of pixels and is split into patches. Those patches are projected into token embeddings, often using a linear map or a convolutional implementation. Positional information is added so the model knows where tokens came from, and optionally a class token is prepended for image-level prediction.

The sequence then passes through stacked transformer encoder blocks, where self-attention and MLP layers repeatedly refine the token representations. Finally, an aggregation choice is made, such as reading the class token or pooling patch tokens, and a task-specific head maps the representation to class logits or another target space.

The main design choices include patch size, embedding dimension, positional encoding type, use of a class token, encoder depth, number of attention heads, and the form of the output head.